


```
// controll ui_function
const bdi_UI_controll_setup = {

    bdi_active_object: {
        active: false,
        test_active: false,
        activeForm: false,
        skipBack: false,
        criticalFlag_suizid: false,
    },

    bdi_setup: {
        "indexBDI": 0,
        "questionLength": bdi_questionLength,
        "display_container": null,

        "btn_click_i": 0,
        "buffer_clickEvent": 1,
        "inputValue": null,

        "bufferPoints": null,
        "points": 0,
    },

    copy_bdi_setup: null,

    // das evaluationSystem ist das auswertung system der Punkte sie soll das mit den hinterleg

    bdi_displaySection_UI(keyWord, metaObject) {

        const parentDom = document.querySelector('.display_bdi');
        switch (keyWord) {
            case 'skipForward':
                renderingFunction_object.removeChildFunction(parentDom);
                this.function_first_initiation(parentDom);
                this.initiation_label_function();
                break;

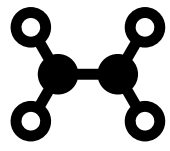
            case 'skipBack':
                renderingFunction_object.removeChildFunction(parentDom);
                this.function_first_initiation(parentDom);
                this.initiation_label_function();

                break;
            case 'evaluation':

                if (!metaObject) { return };
                renderingFunction_object.removeChildFunction(parentDom);
                renderingFunction_object.rendering_functionSystem(metaObject, parentDom);
```

DAS BDI-II PROJEKT 3

Index.html 4



DAS BDI-II PROJEKT

In diesem Projekt präsentiere ich eine vollständige BDI-II-Testanwendung, die ich komplett ohne Frameworks in Vanilla JavaScript entwickelt habe. Seit Januar 2025 erarbeite ich mir täglich intensiv die Softwareentwicklung. Da bei mir eine schwere Legasthenie diagnostiziert wurde, war das Durcharbeiten umfangreicher Fachliteratur – wie *„JavaScript – Das umfassende Handbuch“* von Philip Ackermann – anfangs eine enorme Hürde. Durch eiserne Disziplin und die Kombination mit praxisnahen Plattformen wie W3Schools habe ich mir nicht nur ein tiefes Code-Verständnis, sondern auch das nötige technische Englisch angeeignet. Dieses Projekt steht stellvertretend für meinen Ehrgeiz und mein Durchhaltevermögen.

Warum genau dieses Projekt für meine Bewerbung? Mich hat die komplexe Logik hinter dem Auswertungssystem gereizt. Während mein erster Prototyp noch auf einem einfachen „Weiter“-Button und einer endlosen Kette von ***if***-Abfragen ohne Rückwärtsnavigation basierte, nutzt die aktuelle Version eine saubere, datengesteuerte Architektur. Zudem habe ich einen Sicherheitsmechanismus (***criticalFlag***) integriert: Werden bei der Frage zu Suizidgedanken kritische Punktwerte (2 oder 3 Punkte) gewählt, registriert das System dies und gibt in der Auswertung gezielt Hilfsangebote sowie die Kontaktdaten der Telefonseelsorge aus. Die direkte ***href***-Anruffunktion wurde implementiert, ist in der Vorführversion jedoch bewusst auskommentiert.

In dieser Dokumentation erläutere ich den Aufbau meiner Daten- und Logikmodule. Um meinen persönlichen Entwicklungsprozess sichtbar zu machen, stelle ich dabei auch ältere Code-Versionen gegenüber – vom ersten rohen Entwurf bis zur finalen Engine.

Für mich ist diese Präsentation auch wichtig, um offen zu zeigen, wer ich bin und mit welchen Herausforderungen ich im Alltag arbeite. Viele würden vielleicht raten, eigene Schwächen in einer Bewerbung zu verstecken – für mich liegt wahre Größe jedoch genau darin, offen damit umzugehen. Es zeigt, wer ich bin und wie weit ich gehe, um über mich hinauszuwachsen.

INDEX.HTML

```html
<!DOCTYPE html>
<html lang="de">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover">
    <!-- Passt die Safari-Browserleisten farblich an dein dunkles Design an -->
    <meta name="theme-color" content="#12151b">
    <meta name="apple-mobile-web-app-status-bar-style" content="black-translucent">
    <title>BDI II Test</title>

    <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlined" rel="stylesheet">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Share+Tech&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
    <link href="https://fonts.googleapis.com/css2?family=Belanosima:wght@400;600&family=Varela+Round&display=swap" rel="stylesheet">

    <link rel="stylesheet" href="bdi_mainSection.css">
    <link rel="stylesheet" href="bdi_header_css.css">
    <link rel="stylesheet" href="bdi_intro.css">
    <link rel="stylesheet" href="leftSide.css">
    <link rel="stylesheet" href="instruction.css">
    <link rel="stylesheet" href="bdi_test.css">

    <link rel="stylesheet" href="bdi_mobilMainSection.css">


</head>

<body class="grid grid_setupBody bdi_body_section body_sectionStyle">

    <script src="bdiJSON.js"></script>
    <script src="intro_bdi.js"></script>
    <script src="bdi_btn_code.js"></script>
    <script src="question_bdi.js"></script>
    <script src="bdi_test_ui_Logic.js"></script>
    <script src="renderringBDI_II_function.js"></script>

</body>

</html>
```

Hier ist mein HTML-Code – dies ist meine Leinwand. Ich baue über eine UI-Funktion meinen Body auf, welcher über *children* so lange rekursiv aufgerufen wird, bis die gesamte Struktur vollständig abgearbeitet ist.

Für dieses Verfahren habe ich ein System entwickelt, das robust auf unkritische Abweichungen reagiert und diese kontrolliert abfängt. Zukünftig möchte ich hier noch gezielte ***console.error()***-Ausgaben integrieren, um im Falle von kritischen Fehlern (***critical failures***) sofort einen präzisen Nachweis in der Konsole zu haben.

Man könnte die Haupt-DOM-Elemente natürlich auch statisch direkt in die ***index.html*** schreiben. Mir ging es bei diesem Projekt jedoch darum, meine eigene Architektur und Herangehensweise zu zeigen – zu demonstrieren, wie ich strukturiere, logisch denke und mit dieser dynamischen Art der Softwareentwicklung umgehe.

Der CSS-Code:

Als Nächstes zeige ich Auszüge aus meinen CSS-Dateien. Was mir hier selbst noch nicht zu 100 % gefällt, ist, dass ich stellenweise noch zu viel redundantes CSS schreibe und Regeln wiederhole. Hier sehe ich noch klares Potenzial, mich weiter zu verbessern und fachlich zu wachsen. Dabei muss ich oft an mein Lieblingszitat von Thomas Alva Edison denken: *‚Ich bin nicht gescheitert. Ich kenne jetzt 1000 Wege, wie man es nicht macht – und es reicht ein einziger Weg, damit es richtig funktioniert.‘* Wissen wächst Schritt für Schritt, und jeden Tag kommt durch die Praxis etwas Neues hinzu

```css
*,
*::before,
*::after {
    box-sizing: border-box;
    padding: 0;
    margin: 0;
}

html {
    font-size: 100%;
    overflow-x: hidden;
    background: #87cefa;
}

body {
    width: 100%;
    min-height: 100vh;
    min-height: 100dvh;
}

.flex_directin_column {
    display: flex;
    flex-direction: column;
}

.flex_directin_column_reverse {
    display: flex;
    flex-direction: column-reverse;
}

.flex_directin_row {
    display: flex;
    flex-direction: row;
}

.flex_directin_row_reverse {
    display: flex;
    flex-direction: row-reverse;
}

.grid {
    display: grid;
}
```

CSS-Reset (*, ::before, ::after): Das Sternchen setzt alle Standard-Abstände des Browsers auf null zurück (box-sizing: border-box), damit das Layout auf allen Systemen identisch berechnet wird.

100vh & 100dvh im Body: vh steht für *Viewport Height* (ideal für Desktop-Browser) und dvh für *Dynamic Viewport Height*. 100dvh passt sich auf Smartphones dynamisch an ein- und ausfahrende Browserleisten an.

CSS Grid & Flexbox: Mit display: grid spanne ich das übergeordnete Raster über benannte Bereiche (grid-template-areas) auf. Innerhalb der einzelnen Container nutze ich display: flex (row / column), um die Elemente flexibel auszurichten.

Mobile First & Responsive Design (bdi_mobilMainSection.css): In der modernen Webentwicklung gilt das Prinzip *Mobile First*. Mir fällt es am Anfang eines Projekts manchmal noch schwer, konsequent mit der mobilen Ansicht zu starten, weil es sich anfangs noch ungewohnt anfühlt – aber auch hier gilt: Übung macht den Meister.

```css
@media screen and (max-width: 768px) {

    html,
    body {
        padding-top: env(safe-area-inset-top);
        padding-bottom: env(safe-area-inset-bottom);
    }

    .grid_setupBody {
        grid-template-areas:
            'header'
            'leftSide'
            'main'
            'rightSide'
            'footer'
        ;
        grid-template-columns: 1fr;
        grid-template-rows: 150px 90px 1fr 70px 70px;
    }

    .bdi_body_section_intro {
        padding: 0.4em;
    }

    .bdi_header_section_intro {
        grid-area: header;
        position: sticky;
        top: 0;
        border-bottom: solid 1px #000000;
    }

    .psychiatry_h1_style {
        font-size: 2.4rem;
    }

    .bdi_leftSide {
        grid-area: leftSide;
        justify-content: center;
        flex-direction: column;
        border-right: none;
        border-bottom: solid 1px #000000;
    }
```

Über @media screen and (max-width: 768px) passe ich das Layout gezielt für kleinere Bildschirme an. Das habe ich in der Mitte des Projekts zunächst vergessen, aber mein Anspruch an mich selbst ist es, beim nächsten Projekt direkt damit zu starten und weiter über mich hinauszuwachsen (***Take it higher***).

Beim mobile.css muss man die Reihenfolge (Kaskade) genau beachten, damit sich Desktop- und Mobile-Regeln nicht ungewollt gegenseitig überschreiben. Ich arbeite erst seit meinem vierten Projekt so und entdecke dabei ständig neue Möglichkeiten. Auch wenn CSS keine klassische Programmiersprache, sondern die Stylesheet-Sprache des Internets ist, hat es mehr auf dem Kasten, als man denkt: Viele Animationen und Layout-Berechnungen kann man heute beruhigt dem Browser überlassen, ohne dafür zwingend JavaScript zu benötigen.

Ich habe die jeweiligen Abschnitte im CSS getrennt, um die Übersichtlichkeit zu fördern. Für meinen aktuellen Stand – den ich mir komplett ohne fremde Hilfe erarbeitet habe – bin ich stolz auf meine Fortschritte. Meine größte Baustelle im CSS sind aktuell noch unnötige Dopplungen sowie eine noch klarere Benennung der einzelnen Klassen (class).

JavaScript

Diese Engine besteht aus vielen kleinen Funktionen, die jeweils nur eine klar definierte Aufgabe haben. So lässt es sich später viel besser auf Fehlersuche gehen und auch andere Entwickler verstehen den Code sofort – ganz nach dem Grundsatz *„Keep it simple“* (KISS). Diese Rendering-Engine kann ich jetzt in kommenden Projekten immer wiederverwenden und schrittweise verfeinern. In meiner bisherigen Lernzeit habe ich vor allem eines gemerkt: Es gibt immer noch schönere und elegantere Wege, Code zu schreiben. Solange das Lernen Spaß macht, hört man nie auf, vom Code fasziniert zu sein. Ich weiß noch nicht genau, wohin mein Weg führt, aber eines ist sicher: Mit Durchhaltevermögen und eisernem Willen lässt sich jede noch so komplexe Herausforderung meistern.

```
// BDI UI_renderingFunction

const renderingFunction_object = {

    removeChildFunction(parentDom) {

        if (parentDom.innerHTML.trim() !== '') {

            parentDom.innerHTML = '';
            return;
        } else {

        };

    },

    destructChild_contentFunction(children, parentDom) {
        if (!children || !parentDom) { return };
        const childObject = bdiLibrary[children] || children;
        this.rendering_functionSystem(childObject, parentDom);
        return;

    },

    createHTML_element_func(htmlTag) {
        if (!htmlTag) { return };
        const html_element = document.createElement(htmlTag);
        return html_element;
    },

    toInsertValue(valueNumber, html_element) {
        if (valueNumber === null || valueNumber === undefined || valueNumber === "") { return };
        console.log(valueNumber, html_element);
        html_element.value = Number(valueNumber);
    },

    toInsert_type(typeSet, html_element) {
        if (!typeSet) { return };
        console.log(typeSet, html_element);
        html_element.type = typeSet
    },

    toInsert_name(nameInput, html_element) {
        if (!nameInput) { return };
        html_element.name = nameInput
    },
```

Dies ist ein Code-Ausschnitt meiner ***renderingFunction_object*** – sie ist das Herzstück meines Programms. Sie wandelt alle Daten-Objekte automatisch in sichtbare DOM-Elemente um und ist so aufgebaut, dass sie bei kleineren Abweichungen nicht sofort abbricht. Noch kann ich nicht jeden erdenklichen Ausnahmefall abfangen, aber genau dieses Feingefühl wächst mit jedem weiteren Projekt und der praktischen Erfahrung.

- **Den vollständigen Quellcode sowie die Live-Demo des BDI-II-Tests finden Sie in meinem GitHub-Repository.**